

ARCHITECTURE DECISION RECORD

Schema Design — Star Schema Migration

HiddenGemMusic | Capstone Project | April 2026

Author: Leena | Status: Accepted | Track: BDA | Supersedes: Decision 1 and Decision 3 of the Database Schema Design ADR (April 2026)

1. Context

The Database Schema Design ADR (April 2026) established a fully normalized 3NF schema as the foundation for the HiddenGemMusic database. That decision was made before ingestion, when the operational characteristics of the database were not yet fully known.

After both datasets were ingested and validated — 28 million rows in ChartEntry — it became apparent that the database is exclusively historical and analytical. Data is written once during ingestion and never updated. Every operation the application performs is a read: aggregation, cross-country comparison, trend analysis, and Hidden Gem scoring. There are no transactional writes after the initial load.

3NF is designed to support safe concurrent writes and eliminate update anomalies. Neither concern applies here. What the schema does need to support efficiently is fast, repeated analytical reads across very large fact data — a workload that star schema is specifically designed for.

This ADR records the decision to migrate the schema from 3NF to a star schema dimensional model, the rationale for that decision, and the resulting state of the system.

2. Decision

Migrate the HiddenGemMusic database schema from 3NF to a star schema. FACT_ChartEntry remains the center of the schema unchanged. The five song-related entity tables from the original schema — Song, Artist, ArtistSong, AudioFeatures, and Album — are replaced by three dimensional tables: DIM_Song, DIM_Artist, and Bridge_SongArtist. All pre-computed summary tables, ChartEntry, Country, and ChartType are unchanged.

3. Rationale

3a. The workload is analytical, not transactional

The original Decision 1 justified 3NF on the grounds that normalization "keeps the data clean, consistent, and efficient for the types of aggregate queries this application requires." In practice, 3NF adds join overhead that works against aggregate query performance at this scale. Every read stored procedure that returns a song title and artist name requires a minimum of four joined tables: Song, ArtistSong, Artist, and either AudioFeatures or Album depending on what is being returned. Across 28 million ChartEntry rows, this overhead compounds.

Star schema trades write efficiency for read efficiency by flattening dimension tables. Since no writes occur after ingestion, this tradeoff costs nothing and reduces every dimension lookup from a multi-table join chain to a single indexed lookup on an integer primary key.

3b. Audio features belong on DIM_Song, not in a separate table

The original Decision 3 separated audio features into a dedicated AudioFeatures table to signal that they are secondary, reach-goal data not required for any MVP feature. While this rationale was sound at design time, it created a structural problem: any stored procedure that returns song data and needs to include audio features — even as optional, nullable output — requires an additional join. In a star schema, all attributes of a dimension entity belong on that dimension table. Audio features are attributes of songs. They are stored as nullable columns on DIM_Song, which is the correct placement in a dimensional model.

3c. Album is display data, not a queryable entity

No stored procedure groups, filters, or aggregates by album. Album name and release date appear in query output as display labels only. Maintaining a separate Album entity table with a surrogate key solely to hold a display string adds a join to every procedure that returns song output. Album name and release date are stored as plain nullable string columns on DIM_Song. Album art is retrieved at display time from the Spotify API using spotify_id and is not stored in the database.

3d. Many-to-many artist relationships require a bridge table

Songs genuinely have multiple artists. This relationship cannot be flattened without data loss or denormalization that would make artist-level queries impossible. A bridge table between DIM_Song and DIM_Artist is the correct dimensional modeling pattern for many-to-many dimension relationships. Bridge_SongArtist serves this role, replacing ArtistSong from the original schema. The is_primary BIT column is replaced by artist_order TINYINT, which records the position of each artist in the source data string rather than asserting a primary/featured distinction that the source data does not explicitly encode.

4. Resulting Schema

DIM_Song

Flat dimension table. One row per unique track. The spotify_id column remains the cross-dataset matching key. All DS2-only columns are nullable — DS1 song rows carry only title and spotify_id. DS2 ingestion enriches existing rows and inserts new ones with all available columns populated.

Column	Type	Nullable	Description
song_id	INT	NOT NULL	PK, IDENTITY. Surrogate primary key.
spotify_id	NVARCHAR(50)	NULL	Spotify track ID. Unique constraint. Parsed from URL in DS1; native in DS2. Nullable where parsing fails.
title	NVARCHAR(512)	NOT NULL	Song title as it appears in the source dataset.
duration_ms	INT	NULL	Duration in milliseconds. DS2 only.
is_explicit	BIT	NULL	Explicit lyrics flag. DS2 only. Converted from string to BIT during ingestion.
album_name	NVARCHAR(512)	NULL	Album title. DS2 only. Plain string — no surrogate key, no FK relationship.

Column	Type	Nullable	Description
album_release_date	DATE	NULL	Album release date. DS2 only. Partial dates normalized to first of month or first of year during ingestion.
danceability	FLOAT	NULL	Suitability for dancing. Range 0.0–1.0. DS2 only.
energy	FLOAT	NULL	Intensity and activity. Range 0.0–1.0. DS2 only.
key	TINYINT	NULL	Pitch class notation, 0–11. -1 if no key detected. DS2 only.
loudness	FLOAT	NULL	Overall loudness in dB. Typical range -60 to 0. DS2 only.
mode	TINYINT	NULL	1 = major, 0 = minor. DS2 only.
speechiness	FLOAT	NULL	Presence of spoken words. Range 0.0–1.0. DS2 only.
acousticness	FLOAT	NULL	Acoustic quality confidence. Range 0.0–1.0. DS2 only.
instrumentalness	FLOAT	NULL	Likelihood of no vocals. Values above 0.5 are likely instrumental. DS2 only.
liveness	FLOAT	NULL	Live audience presence. Values above 0.8 indicate a strong likelihood of live performance. DS2 only.
valence	FLOAT	NULL	Musical positiveness. High = happy, low = sad. Range 0.0–1.0. DS2 only.
tempo	FLOAT	NULL	Estimated tempo in BPM. DS2 only.
time_signature	TINYINT	NULL	Estimated beats per bar. Range 3–7. DS2 only.

DIM_Artist

One row per unique artist name. artist_id is the stable integer primary key used by Bridge_SongArtist. artist_name is stored as NVARCHAR(MAX) to accommodate the full range of artist string lengths present in both datasets, including multi-artist packed strings in Dataset 1 and multi-byte character sets. A unique index on artist_name is not possible for MAX-length columns in SQL Server; uniqueness is enforced during ingestion via NOT EXISTS logic.

Column	Type	Nullable	Description
artist_id	INT	NOT NULL	PK, IDENTITY. Surrogate primary key.
artist_name	NVARCHAR (MAX)	NOT NULL	Artist display name as it appears in the source datasets. No unique index — SQL Server cannot index MAX-length columns.

Bridge_SongArtist

Resolves the many-to-many relationship between DIM_Song and DIM_Artist. One row per song-artist pairing. artist_order records the position of each artist in the source data string — 1 identifies the first-listed artist and is used by stored procedures and the API layer as the primary artist for display purposes.

Column	Type	Nullable	Description
song_id	INT	NOT NULL	FK -> DIM_Song. Composite PK with artist_id.
artist_id	INT	NOT NULL	FK -> DIM_Artist. Composite PK with song_id.
artist_order	TINYINT	NOT NULL	Position of this artist in the source data string. 1 = first-listed artist. Used by stored procedures and the API layer to identify the primary display artist.

Unchanged Tables

The following tables are unchanged from the original schema. Their structure, data, and indexes are identical to what was documented in the Database Schema Design ADR.

Table	Notes
FACT_ChartEntry	Unchanged. The song_id foreign key now references DIM_Song instead of Song, but all 28 million existing rows are intact and song_id values are identical.
Country	Unchanged.
ChartType	Unchanged.
HiddenGems	Unchanged. Foreign key on song_id updated to reference DIM_Song.
SongCountryPresence	Unchanged. Foreign key on song_id updated to reference DIM_Song.
CountryYearStats	Unchanged.
GlobalOverlapByYear	Unchanged.
DiscoveryGapByDay	Unchanged. Foreign key on song_id updated to reference DIM_Song.
IsolationScoreByCountry	Unchanged.
PeakReachBySong	Unchanged. Foreign key on song_id updated to reference DIM_Song.
TrendVelocityBySong	Unchanged. Foreign key on song_id updated to reference DIM_Song.

5. Stored Procedure Impact

5a. Population Procedures

All eight population stored procedures are unchanged. They operate exclusively on FACT_ChartEntry and the pre-computed summary tables and do not reference the five replaced entity tables. No modifications were required.

5b. Read Procedures

Seven read stored procedures were updated. The change was structurally identical across all seven: the multi-table join chain for song, artist, and album data was replaced with the star schema equivalents. In all seven procedures, album references that previously joined through a separate Album table now read album_name directly as a string column from DIM_Song. The Album join is eliminated entirely.

Procedure	Nature of Change
sp_GetComparisonHiddenGems	Song, ArtistSong, Artist, Album joins replaced with DIM_Song, Bridge_SongArtist, DIM_Artist. album_name read from DIM_Song.
sp_GetCountryComparison	All five result sets updated with the same join pattern replacement. No change to logic, CTEs, or output shape.
sp_GetCountryHiddenGemsPreview	Song, ArtistSong, Artist, Album joins replaced. album_name read from DIM_Song.
sp_GetCountryProfile	Both the shared songs and unique songs result sets updated. album_name read from DIM_Song.
sp_GetHiddenGems	Song, ArtistSong, Artist, Album joins replaced. album_name and is_explicit read from DIM_Song.
sp_GetPeakCrossRegionalReach	Song, ArtistSong, Artist, Album joins replaced. album_name read from DIM_Song.
sp_GetGlobeSummary	Simplified. The nested subquery that joined Song to retrieve album_id and then joined Album to retrieve album name was replaced with a direct read of DIM_Song.album_name.

Six read procedures required no changes: sp_GetAverageDiscoveryGap, sp_GetDiscoveryGapDistribution, sp_GetGlobalOverlapRate, sp_GetGlobalOverlapTrend, sp_GetIsolationLeader, and sp_GetIsolationRanking. These procedures read exclusively from pre-computed summary tables and do not reference any song or artist dimension table.

6. Trade-offs and Limitations

6a. No unique index on DIM_Artist.artist_name

SQL Server does not support unique indexes on NVARCHAR(MAX) columns. The original Artist.name column had a unique constraint that enforced exactly one row per artist name at the database level. That constraint cannot be re-created on DIM_Artist.artist_name. Uniqueness is enforced during ingestion via NOT EXISTS logic in both ingestion scripts. If the same artist name appears with minor formatting

differences across datasets, duplicate DIM_Artist rows may exist. This is a known limitation of the MAX column type choice and is acceptable for a capstone with a fixed, non-updating dataset.

6b. Album name is denormalized

album_name on DIM_Song is a plain string column. The same album may appear with slightly different formatting across different songs — trailing whitespace, minor punctuation differences, or encoding variations. In the original schema, the Album surrogate key would have caught this if album rows were deduplicated correctly during ingestion. As a plain string, no such deduplication occurs. Because no stored procedure groups, filters, or aggregates by album name — it appears only as a display label — this limitation has no impact on analytical correctness.

6c. Some DS1 artist fields were ingested as single artist strings

Dataset 1 does not consistently delimit multiple artists. Some rows in DS1 contain comma-separated artist names packed into a single artist field that could not be reliably split. These were ingested as single DIM_Artist rows containing the full packed string. They are identifiable by an unusually long artist_name value. They will display correctly in the UI as-is but will not produce individual artist rows for the artists named within them. A post-ingestion cleanup pass is possible but is not required for any MVP feature.

6d. Audio features are DS2 only

Audio feature columns on DIM_Song are NULL for all songs that appeared only in Dataset 1 and were never matched to a Dataset 2 entry. This was also true in the original schema where AudioFeatures rows simply did not exist for DS1 songs. The behavior for stored procedures and the API layer is unchanged — audio feature data must be handled as optional and NULL-safe in any component that uses it.

7. Consequences

- All stored procedures that return song or artist data now execute one fewer join than before. Dimension lookups are single indexed integer key lookups rather than multi-table chains.
- Audio feature data is available directly on DIM_Song for any future stored procedure or API endpoint that needs it. No additional join or table is required.
- Any future feature that requires grouping or filtering by artist must use DIM_Artist and Bridge_SongArtist. The bridge table pattern is permanent — the many-to-many relationship cannot be flattened.
- The .NET 9 API layer is unaffected. All stored procedure output shapes are identical — the API receives the same JSON structure regardless of whether the data came from Song or DIM_Song.
- The pre-computed summary tables, population procedures, and all KPI logic are unaffected by this migration.